

ARPI

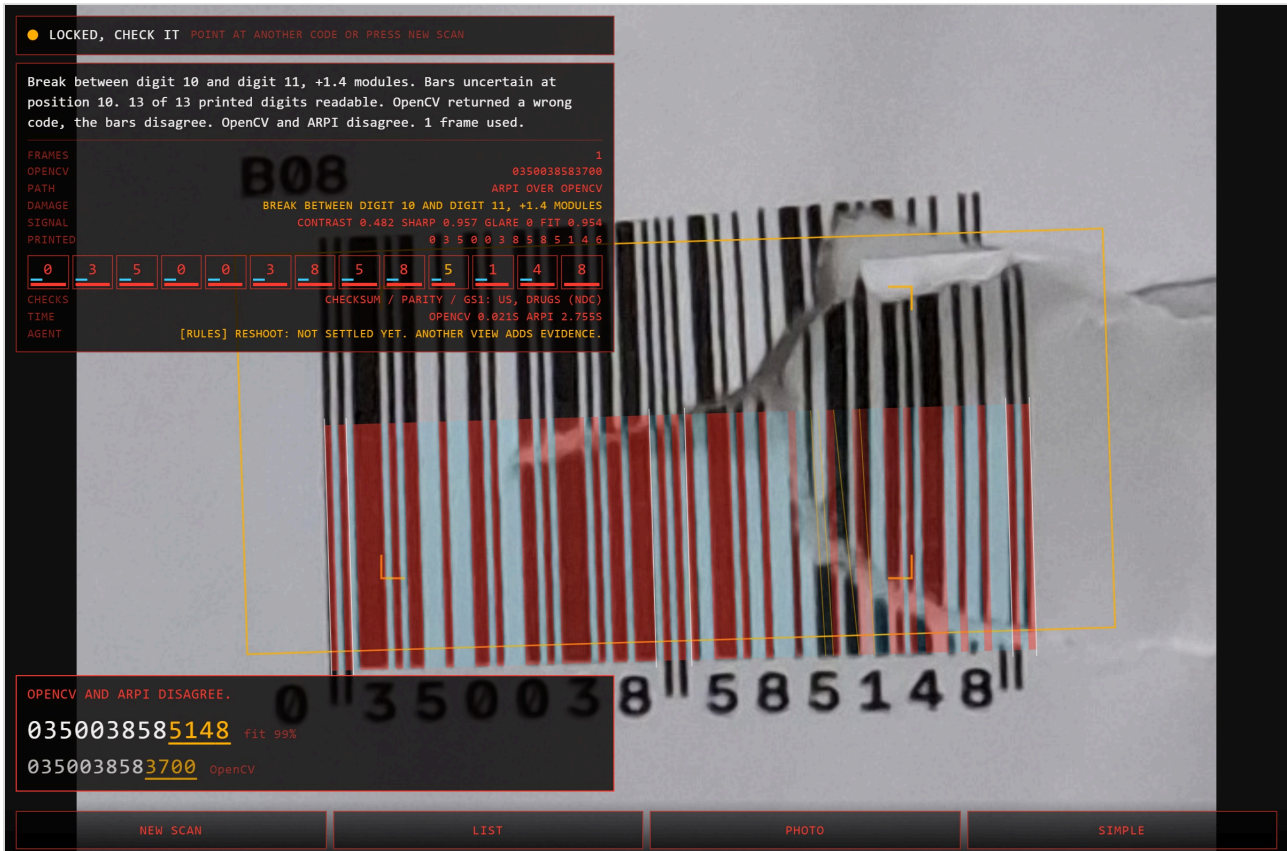
Reads barcodes too damaged to scan. Technical report, OpenCV AI Competition 2026.

Author: Juha Lilja (APN201)

Live: <https://eu2oqybdr2td26obiljef6xopu0jsmwo.lambda-url.eu-west-1.on.aws/>

Code: <https://github.com/apn201/Arpi>

Results quoted here: results/2026-10-05/ in the repo, frozen with tool versions and hashes



A torn code from the test set, run through the scanner page. OpenCV returns 0350038583700, which passes its checksum and is wrong. ARPI finds the tear (break between digit 10 and 11, +1.4 modules), says the bars disagree with OpenCV, and its top candidate 0350038585148 is the printed code.

1. Problem

You try to read a barcode with a phone and it does not read. A label that has been through a warehouse, a torn corner, a strip of tape over it. The scanner beeps, you try again, and after a few tries you type the digits by hand. That was the fallback on the handheld terminals I specified for a printing ink plant in 2006 (my bachelor's thesis, "Implementation of Barcode Functionality in ERP-system"), and it is still the fallback.

An EAN-13 has twelve data digits and one check digit, and that is the whole defence:

$$\text{sum}(w_i * d_i, i = 1..12) + d_{13} = 0 \pmod{10}, w = 1, 3, 1, 3, \dots$$

A normal reader that loses part of the code fails the checksum and gives up. Or worse, it finds a wrong code that passes. One check digit catches nine wrong codes in ten, so a damaged read that is not

rejected can still be wrong, and the output is a drug, a part or a price.

A damaged code is not a random string though. Each digit is seven modules with a fixed shape. The guard bars are fixed. The left half hides the 13th digit in an odd/even parity pattern, and 54 of the 64 possible patterns are illegal. The leading digits must be an assigned GS1 prefix. And in a real warehouse the code is one of the few thousand that exist in the company's ERP. Put together, two destroyed digits usually come down to a handful of candidates, and with a code list often to one.

QR and Data Matrix are out of scope on purpose. They carry Reed-Solomon error correction and already recover from serious damage by themselves.

2. Users

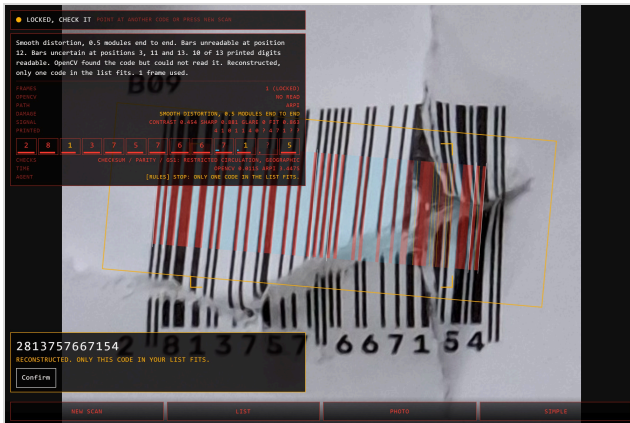
Anyone who scans 1D codes for a living and loses time on the ones that do not read: warehouse and goods-in staff, stock takes, returns, a pharmacy counter, a shop till. Also the occasional person with a phone and a crumpled receipt. The product is a web page, so there is nothing to install: open the link, point the camera at one code, hold still.

The deployment I had in mind is plain. The customer drops a CSV of their valid codes next to it, exported from the ERP. No connector, no integration. It also works with no list at all, and then it says so when the answer is not unique.

3. What it does

- OpenCV's own barcode decoder goes first. It is fast and reads the easy codes.
- ARPI takes what OpenCV cannot read. It also checks what OpenCV does read against the bars, because OpenCV is sometimes confidently wrong (section 9).
- Every frame adds evidence. Glare and blur move when the phone moves, so the next frame fills in what the last one hid.
- After each frame a small agent decides: stop, or ask for a specific better shot, and it cites the measurement behind the request.
- The answer is one of four things: a clean read, the only code in your list that fits, ranked candidates, or just the digits that were readable.

The safety rule: ARPI never asserts a code it had to reconstruct. A reconstruction is shown as options with their fit and the person picks. When the list leaves only one code, the page says "Reconstructed. Only this code in your list fits" and still asks the person to confirm. The system selects, the human confirms.

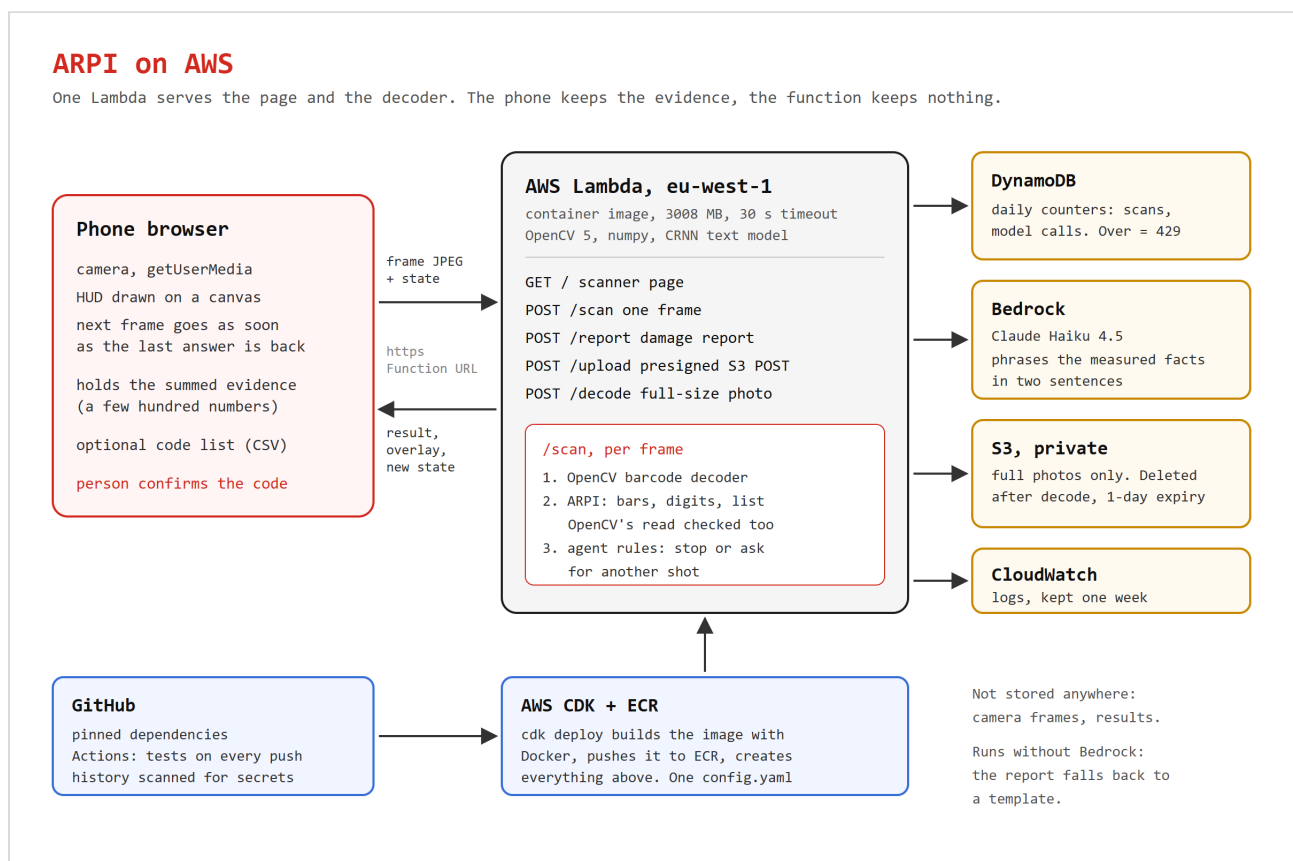


Torn and distorted, three digits unreadable from the bars. With the list loaded only one code fits. It is the right one, and it still needs Confirm.



The leading digit is under a piece of paper. The bars carry it only through the left half's parity pattern, which is enough.

4. Architecture

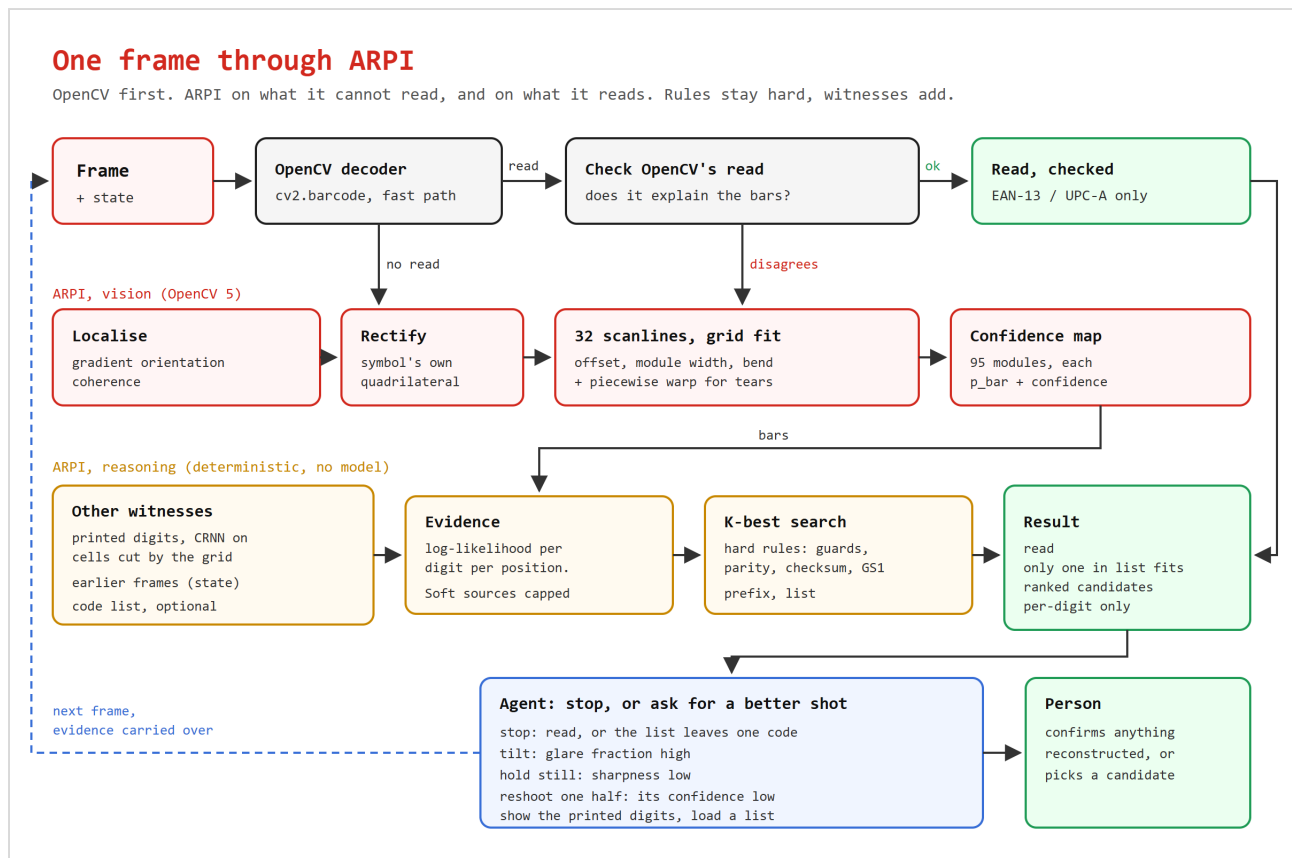


The deployed system. One Lambda container serves the page and the decoder. The phone holds the accumulated evidence and sends it back with each frame, so the function is stateless.

The phone side is a single HTML page with no framework: camera through getUserMedia, a canvas HUD, and a loop that sends the next frame as soon as the last answer is back. The server side is one Lambda function behind a Function URL. Serving the page from the same function gives the phone https, which a camera needs, without a separate web host.

The function holds nothing between requests. The evidence for the code being scanned is a few hundred numbers (a log-likelihood per digit per position). The phone keeps it and sends it back with the next frame. That is what makes a multi-frame scanner fit on Lambda.

5. OpenCV 5 implementation



One frame. Everything left of the confidence map looks at pixels; everything after it does not.

The cascade

`cv2.barcode.BarcodeDetector` goes first. Its boxes are reused when it found a symbol it could not read. Its reads are checked before they are accepted: anything that is not EAN-13 or UPC-A goes to ARPI (in this set the commonest wrong read is an EAN-8 read out of part of an EAN-13), and the read is accepted only if it explains the bars nearly as well as ARPI's best candidate.

Vision

- **Localise.** Gradient orientation coherence: bars give a strong single direction over an area, which generic contour finding does not see under damage. Two bugs the real photos found here: the paper edge is a closed loop of coherent gradient and hid every code inside it, and the torn-fragment merger once chained twelve codes and a keyboard into one region.
- **Rectify.** Perspective correction from the symbol's own quadrilateral, so module widths can be compared at all.
- **Grid fit on 32 scanlines.** Each line gets an offset, a mean module width and a bend term (a label tilted away from the camera has modules growing from one end to the other). The fit is scored by asking whether every digit looks like some legal digit, not by finding edges. That is why it survives destroyed guard bars. Module means are integrated from a cumulative sum, so it works at sub-pixel precision.
- **Piecewise warp.** Wrinkles and torn pieces that were put back a bit wrong are followed by splitting the grid where the fit says it broke. The acceptance margin was set from an end-to-end comparison, not guessed.

- **Confidence map.** 95 modules, each with two numbers: how dark it looked and how much to believe that. They are never collapsed into one. A sticker reads as a confident white run, and confidence is what lets the next stage say "seven white modules in a row never happens in EAN-13, do not trust this".

Printed digits

The digits under the bars are a second, independent witness, read with `cv2.dnn` and the CRNN text model from OpenCV Zoo (pinned by hash). The bar grid already says which columns hold each digit, so each cell is cut out by geometry and the cells go to the network as two clean strips. No text detection. Orientation comes from the bars, never from the text: upside-down digits read as confident nonsense.

6. Reasoning

Every source becomes the same thing: a log-likelihood for every digit at every position, with the left half split by parity because the bars can tell the two digit sets apart and nothing else can. Sources add up. They do not vote on finished answers, because a vote would make the checksum one opinion among several.

The search is an exact K-best over checksum residue and parity, so a fully destroyed digit costs nothing extra. The structural rules (guards, parity, checksum, GS1 prefix) stay hard inside it, so no source can produce a code the rules forbid. Each soft source is capped so it cannot outvote clearly read bars, and that ceiling has its own test. Frames of the same label are not independent (the same smudge is in all of them), so n frames are scaled by n to the power -0.5 rather than trusted n times.

With a code list, the list is scored directly against the evidence. If one list code is clearly ahead of every other, the answer is "only one code in your list fits". If two fit about equally, both are shown.

7. The agent

After each frame the agent reads the scan's measurements (glare fraction, sharpness, contrast, per-half confidence, what the printed digits gave, how the code resolved) and returns one decision with the evidence it acted on:

decision	when
aim	no code in view, or the frame was skipped (bad grid fit, low contrast)
stop	read and checked
stop, confirm	only one code in the list fits
stop, pick one	many frames and still no single answer: more of the same view will not help
reshoot: tilt	share of the code washed white by glare over the limit
reshoot: hold still	sharpness under the limit
reshoot: one half	the halves differ ("right half confidence 0.38, other half 1.00")
show the digits	the bars give nothing at some positions and the printed digits have not been read there
load a list	more than one code fits what is visible

reshoot: other none of the above: another view adds evidence
angle

Each request changes what the person does next, and the next frame's evidence is added to what is already known. That is the loop: an OpenCV measurement leads to a request, the request leads to a different picture, the picture changes the answer.

The rules are deterministic and are the default. I also wired Claude on Bedrock to the same decision, given the same measurements and the rules' proposal, and required to cite what it acted on. In live use it added long, often pointless comments on codes that were already read, every one a model call, and it made the same decisions as the rules. So it is switched off and kept switchable. Bedrock (Claude Haiku 4.5) does one job in the deployed version: it phrases the measured damage facts in at most two sentences when a scan locks. The facts are extracted in code. Model output that gives advice or runs long is discarded and the template version of the same facts is shown.

8. AWS deployment

- **Compute.** Lambda container image (OpenCV, numpy and the text model pass the 250 MB zip limit), 3008 MB, 30 s timeout, eu-west-1, behind a Function URL.
- **Repeatable.** One `cdk deploy` builds the image with Docker, pushes it to ECR and creates the function, bucket, table, log group and IAM policy. `infra/config.yaml` is the only place a region, model or limit is named. Dependencies are pinned in `requirements.txt` and the Dockerfile.
- **Observable.** CloudWatch logs, kept one week. Every result carries which path produced it (OpenCV checked, ARPI over OpenCV, ARPI alone) and the timing of each stage, and the HUD shows them.
- **Secure.** IAM limited to the one log group, the one bucket, the one table and Bedrock model invocation. S3 bucket private with all public access blocked. No account id in the repo; CDK reads it from the profile. A script scans the whole git history for keys before anything is pushed. GitHub Actions runs the tests on every push without AWS credentials.
- **Cost and abuse.** A public URL is an open invitation, so limits are in code, not only in AWS Budgets: upload size, list size and model calls per request, and daily counters in DynamoDB for scans and model calls. Over the ceiling the endpoint says so (HTTP 429). It never fails open.

9. Evaluation

Dataset

The dataset was built first. 72 EAN-13 codes from random GS1 prefixes, printed on six A4 sheets at nominal size (0.34 mm module), each with an id printed beside it. Then damaged by category: scratch, tear, crumple, fade, ink smear, tape and objects over the codes, glare through a plastic sleeve, motion blur. Photographed with a Samsung phone at 4080 x 3060. Ground truth is exact because I printed it. 23 photos; a manifest records each file's hash, sheet and damage.

Codes are matched to their truth by OpenCV's reads, by the printed id and by the 2 x 6 layout, never by ARPI's answer. Two runs:

- **Sheets:** every code in the whole-sheet photo, 240 slots in 20 photos. The two motion-blur photos and the one where fingers cover most codes could not be anchored and are not scored.
- **Singles:** each code cut out with a close-up's margin and run through exactly the path the phone runs, one frame. 225 crops at about 4.7 px per module, a phone held about 30 cm away. 15 of them

caught more than one label (angled photos, the margin reaches the next code) and are not scored, which leaves 210.

"Asserted" means the system committed to one code: a clean read, or "only one code in the list fits". Ranked candidates are not an assertion. The row I watch most is asserted and wrong.

Whole sheets, 240 codes

	OpenCV	ARPI alone	ARPI + list	cascade, checked	cascade + list
right code first	152	210	229	212	229
asserted	154	110	228	168	226
asserted and wrong	2	0	0	0	0

The cascade without the check (trust OpenCV) asserts 169 right and 2 wrong. The free symbology check removes one; the full check against the bars removes both, at the cost of one right read and ARPI's time on every symbol.

Single codes, 210 crops, the phone's path

	OpenCV	ARPI alone	ARPI + list	cascade	cascade + list
right code first	160	188	196	186	194
asserted	161	103	194	162	190
asserted and wrong	1	0	0	0	0

The 15 crops left out are not clean either. In one of them, with the list loaded, ARPI answered with the code of the label above the one aimed at (A02 for A04, both in the crop). That is a real code in the picture, but not the one the person pointed at, and it is the kind of mistake the phone's aiming (the symbol nearest the centre) is there to prevent. An earlier run of the same set, before the last fixes, had three wrong "only one code in the list fits" answers on steep angles and glare; they now come back as candidates. Any answer of that kind is shown as "Reconstructed, only this code in your list fits" with a Confirm button, never as a plain read.

By damage, sheets, no list

damage	codes	OpenCV right	ARPI first	ARPI asserted	ARPI wrong
clean (straight + angled)	144	98	126	85	0
glare	24	11	20	6	0
tear	24	17	21	3	0
crumple	12	9	11	2	0
fade	12	8	10	6	0
smear	12	7	12	8	0
tape	12	2	10	0	0

Without a list ARPI asserts little on damaged codes and ranks the truth first on most of them. That is the rule working: a reconstruction is a candidate list. The list is what turns it into an answer.

Synthetic

Rendered labels, labelled synthetic wherever quoted. On 224 images over ten damage kinds OpenCV read 144, ARPI ranked the right code first on 186 and asserted 85, none wrong. An earlier synthetic run covered the hard case the physical set does not have: a list of 1000 codes from one company prefix, where neighbours differ in one digit. On 48 tears and occlusions of 6-20% width OpenCV read 10, ARPI asserted 35 with 0 wrong and returned the rest as candidates.

Failure cases



Tape over a third of the code, no list loaded. 27 of 95 modules masked, 32 codes fit. ARPI does not answer. The true code, 9120547365484, is second at 19%. The agent asks for a reshoot of the right half.

- Motion blur: both motion photos had nothing readable to anchor the sheet. Not scored, and not solved.
- Translucent tape lowers contrast without lowering ARPI's confidence enough, so faint bars under it can read as confidently wrong. They come back as candidates, but the truth is not always first.
- A crop holding two labels: with the list, ARPI once answered the neighbour's code.
- Heavy displaced tears (a piece moved more than 3 modules, or overlapping and hiding bars) stay mostly unread in the synthetic set: 2 of 8, OpenCV 0.

10. Limitations

- EAN-13 and UPC-A only. Code 128, GS1-128 and Code 39 are not implemented.
- The physical set is small: 72 codes, one phone, one person, one day, indoor light. The numbers say what happened on it, not more.
- The list in the physical test is 72 codes from random prefixes, which is forgiving.
- Rectification assumes a flat label. A long code wrapped round a can is not handled.

- Single crops are at about 4.7 px per module. A code filling the frame at close range has not been measured on its own.
- The agent's requests are tested in code and used live, but how often a request turned a failure into a read has not been measured.
- The demo's daily scan ceiling can run out under heavy use, and then it returns 429 until midnight UTC.

11. Responsible use

- **Never assert a reconstruction.** A code that had to be filled in is shown as candidates with their fit, or as the only list code that fits, and a person confirms. This is in the UI, not a footnote. The false positive rate is measured and published above, including the cases where it was not zero.
- **No storage.** Camera frames are decoded in memory and not stored. Full-size photos go through a private bucket, are deleted after decoding, and anything left behind expires in a day.
- **No personal data in the test set.** The physical set is printed test codes. Photos stay out of the repo; their hashes are in the manifest.
- **The model does not decide.** Decoding and the candidate engine are deterministic. The model only phrases measured facts, and its output is checked and replaced if it advises or runs long.
- **Bounded cost.** Hard limits in code per request and per day, so a public link cannot run up a bill.
- **Runs without the network.** The decoder and the code list work offline (`python -m arpi photo.jpg --known codes.csv`), for sites that will not let anything leave.

12. Prior art

Damaged-barcode reconstruction is patented in several places: Intermec's permutation-based reconstruction of irregularly damaged codes (US6039252, 1998), self-checkout systems that predict the full code from a partial scan and offer ranked candidates, parcel networks that match a damaged code against candidates using shipper and date, Brady's machine-learning diagnosis of why a decode failed (US12001916) and Socure's generative enhancement of barcode images (US11875259). Commercial SDKs also read damaged and curved codes. That says the problem is worth money. I did not find an earlier system that puts machine vision, an exact constraint search and an agent that asks for a better photo together, open, with its false positive rate published.

13. Reproduce

```

pip install -r requirements.txt
python tools/fetch_models.py           # CRNN text model, pinned by hash
python -m pytest                       # 63 tests
python tools/serve_local.py            # scanner page at http://localhost:8013
python -m arpi photo.jpg --known codes.csv
python tools/eval_sheets.py data/physical/photos --cascade
python tools/eval_singles.py data/physical/photos
python tools/evaluate.py --n 8         # synthetic sweep

aws sso login --profile <profile>
cd infra && npx cdk deploy --profile <profile>

```

The physical photos are not in the repo (size, and the background of a few shows things that are not mine to publish). The manifest lists each file with its SHA-256.

14. Next

Two problems from my own use, both Code 128. GS1-128 labels on our 1 kg ink tins at work are long and curve round the tin, and a static scanner sometimes cannot read them at all. With a streaming camera you can turn the tin, or the camera, and stitch the parts across frames. And Finnish invoice barcodes for payments, read from a pdf on a pc screen. The bank barcode has its own stack of checks (IBAN, reference number, due date), and a payment is exactly where a wrong code must never be asserted.